Übungsblatt 3: Interrupts und Timer

Mikrocomputertechnik 3 · Zustände, Zeitraster und Kontext auf Papier

Ziel und Arbeitsweise

Sie analysieren Trap-Zustände, Einsprungadressen, Timer und externe Quellen. Alle sechs Pflichtaufgaben sind ohne Simulator oder Laborplatine lösbar. Annahmen: ein Hart im Machine Mode, RV32, keine Delegation, zunächst kein Nesting. RV32 hat 32-Bit-Integerregister; ein Hart ist ein Hardware-Ausführungskontext.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht sind 55 Minuten angeleitete Arbeitsphasen vorgesehen: Aufgabe 1 (10 min), Aufgaben 2–3 (15 min), Aufgaben 4–5 (15 min), Aufgabe 6 (15 min). Lösungsbesprechungen gehören zu den Vorlesungsphasen.

Geben Sie zu Rechnungen Einheit, Rechenweg und Interpretation an. Hexadezimale Werte beginnen mit `0x`. Alle Werte sind didaktische Annahmen, keine Messungen.

Aufgabe 1: Trap-Zustand und Freigaben (6 Punkte)

Vor einem Timer-Trap gilt: Hauptprogramm im Machine Mode, globale Freigabe `mstatus.MIE = 1`, Timerfreigabe `mie.MTIE = 1`. Die Hardware schreibt den Rohwert `0x80000007` nach `mcause`. MIE bedeutet Machine Interrupt Enable, MTIE Machine Timer Interrupt Enable.

1. Trennen Sie Interrupt-Bit und Cause-Code. Was bedeutet der Code? Was bedeutet stattdessen `0x00000002`? (2 P)
2. Beschreiben Sie MIE und MPIE unmittelbar nach dem Eintritt sowie nach einer korrekten Rückkehr mit `mret`. MPIE ist Machine Previous Interrupt Enable. Ändert `mret` dabei automatisch `mtimecmp`? (2 P)
3. Der Timer ist später pending, MTIE ist 0, MIE ist 1. Wird im Modell ein Timer-Trap genommen? Hält diese Maskierung `mtime` an? Begründen Sie. (2 P)

Ihre Begründung:

Aufgabe 2: Vektoradressen und Geräte-ID (5 Punkte)

Die Basis ist `BASE = 0x80000100`. `mtvec` besteht aus BASE und MODE in den untersten zwei Bits. Die Basis erfüllt die Ausrichtungsanforderungen des Modells.

1. Welcher `mtvec`-Registerwert stellt Vectored, MODE 1, ein? (1 P)
2. Geben Sie die Einsprungadressen für Timer-Interrupt Cause 7, External-Interrupt Cause 11 und Exception-Code 2 an. Begründen Sie. (3 P)
3. Der PLIC hat zufällig eine externe Quelle mit Geräte-ID 7. Benutzt sie automatisch den Timer-Vektorslot? (1 P)

PLIC bedeutet Platform-Level Interrupt Controller. Ein Slot enthält einen ausführbaren Befehl; die Tabelle ist keine automatisch geladene Zeigertabelle.

Ihre Rechnung:

Aufgabe 3: Timerfrequenz und RV32-Zugriffe (5 Punkte)

Timerfrequenz 10 MHz, Intervall 1 ms, `mtime` und `mtimecmp` jeweils 64 Bit. Für die Teilzugriffe gilt Little Endian und starke Gerätezugriffsordnung.

1. Wie viele Timer-Ticks benötigt das Intervall? Würde eine Prozessorfrequenz von 100 MHz daran etwas ändern? (1 P)
2. Der Zähler geht von `0x00000001_FFFFFFFF` zu `0x00000002_00000000`. Software liest HI 1 vor dem Übergang und LO 0 danach. Welcher falsche Gesamtwert entsteht? Wie erkennt HI–LO–HI den Übergang? (2 P)
3. Der alte Vergleich ist `0x00000001_FFFFFFF0`, der neue `0x00000002_00000010`. Die aktuelle Zeit sei `0x00000001_FFFFFF80`. Warum wäre „neues LO zuerst, dann neues HI“ ungünstig? Geben Sie die geeignete Dreischrittfolge an. (2 P)

HI und LO bezeichnen obere und untere 32-Bit-Hälfte.

Ihr Rechenweg und die Zugriffsfolge:

Aufgabe 4: Sollraster und verspätete Behandlung (5 Punkte)

Timer 10 MHz, Periode $\Delta = 10000$ Timer-Ticks, nächster Solltermin $d = 100000$ Ticks. Im betrachteten Zeitraum gibt es keinen Zählerumlauf und keinen arithmetischen Überlauf.

1. Die Behandlung liest $n = 102000$ Ticks. Vergleichen Sie den nächsten Termin bei `now + 10000` und beim absoluten Sollraster. Geben Sie beide Zeiten in ms an. (2 P)
2. In einem getrennten Fall liest die Behandlung erst $n = 153000$ Ticks. Bestimmen Sie die Zahl fälliger Solltermine und den nächsten Zukunftstermin. (2 P)
3. Warum darf eine Variable „vergangene Millisekunden“ nicht einfach bei jedem Handler-Eintritt um eins steigen? (1 P)

Ihre Rechnung:

Aufgabe 5: PLIC-Zustände nachvollziehen (6 Punkte)

Ein Machine-Mode-Kontext, alle Quellen freigegeben, alle anfangs pending:

ID	Quelle	Priorität
1	UART	3
2	Sensor	7
3	Taster	7

Threshold 5. UART ist eine serielle Schnittstelle. Die Hardware bevorzugt bei gleicher Priorität die kleinere ID. Ein Claim ist ein expliziter Übernahmezugriff und wird nicht vom Threshold gefiltert.

1. Besteht anfangs eine externe Benachrichtigung? Welche ID wird zuerst geclaimt? Warum nicht ID 3? (2 P)
2. Software führt insgesamt vier Claims aus, auch wenn zwischenzeitlich keine Benachrichtigung mehr besteht. Nach jeder Nichtnull-ID bearbeitet sie die Geräteursache vollständig und meldet Complete derselben ID. Es entstehen keine neuen Ereignisse. Füllen Sie die Tabelle. (2 P)

Claim	zurückgegebene ID	noch pending danach	Benachrichtigung nach Bearbeitung
1			
2			
3			
4			

3. Was ist bei ID 0 zu tun? Was kann bei einer Level-Quelle passieren, wenn Software nur Complete meldet und die Geräteursache nicht beseitigt? (2 P)

Ihre Erläuterung:

Aufgabe 6: Geteilter Zustand und Nesting (5 Punkte)

1. Ein vorzeichenloser 32-Bit-Millisekundenzähler hat `last = 0xFFFFFFF0` und `now = 0x00000020`. Berechnen Sie die Differenz modulo 2^{32} und nennen Sie ihre Grenze. (1 P)
2. Hauptprogramm liest Flag 1, eine ISR setzt es erneut auf 1, dann löscht das Hauptprogramm auf 0. Bleibt die zweite Markierung erhalten? Ändert `volatile` diesen Ablauf zu einer atomaren Gesamtaktion? (1 P)
3. Nur für ISR-Frames stehen 256 Byte bereit, je Frame werden im Modell 64 Byte benötigt. Wie viele Frames passen höchstens? (1 P)
4. Beim äußeren Trap wird `mepc = 0x80001000` gesetzt. Software gibt Interrupts wieder frei, bevor sie `mepc` sichert. Ein innerer Trap schreibt `0x80002040` nach `mepc`. Welche Rückkehrinformation fehlt, und was muss vor der Wiederfreigabe grundsätzlich geschehen? (2 P)

ISR bedeutet Interrupt Service Routine. Ein Frame ist ein Software-Sicherungsrahmen. Das Budget in Teil 3 schließt alle anderen Stackverwendungen bereits aus.

Ihre Antworten:

Optional: reine Rechnung oder konkrete Simulation

Die C-Funktion `next_future` aus den Folien kann auf dem Host für die Fälle `(100000, 99999, 10000)`, `(100000, 100000, 10000)` und `(100000, 153000, 10000)` geprüft werden. Erwartete Ergebnisse sind `100000`, `110000` und `160000`. Das prüft keine Geräteadresse.

Eine Renode-Untersuchung benötigt dagegen eine vorhandene, konkret benannte Plattform und passende Firmware samt Trap-Eintritt. Dokumentieren Sie Timerfrequenz, Freigaben und tatsächlich beobachtete Zustände. Simulatorzeit und Hostzeit sowie Modellprüfung und reale Chiplatenz bleiben verschiedene Nachweise.